

JMPM

Changelog

DATA
07/10/2026

ARQUIVO
CHANGELOG.md

Todas as mudanças relevantes no projeto AdvPP.

[4.4.6] - 2026-10-07

Added

- **advplc check --format json (M1).** Um objeto JSON por linha (`{"file":..., "ok":true|false, "error"?..."}`), ordem estável de entrada, erros no próprio JSON via stdout, exit 0/1 igual ao humano, formato desconhecido falha alto. Pensado para agentes (PiG/pig-advpp) e CI parseável; modo humano inalterado. Testes: `TestRunCheckJSONMixed`, `TestRunCheckJSONAllOK`, `TestParseOptionsFormat` (`cmd/advplc/check_format_test.go`).
- **Botão "Perguntar ao PiG" no advpp-ide (Passo 3).** Menu Tools → diálogo → `pig -p --mode json --no-session` em goroutine com o arquivo em contexto (teto 12k chars), resposta final extraída do JSONL (`message_end` autoritativo, `turn_end` fallback) no console. Config por `ADVPP_PIG_BIN/MODEL/TIMEOUT_SECS`; sem `-a` automático (trust é do usuário). `OutputConsole.Append` com mutex (fyne 2.4.4 não tem `fyne.Do`). Testes: parser (incl. contra saída real do pig), prompt-truncate (`cmd/advpp-ide/agent_test.go`).
- **Teto de geração do LLM configurável.** Não havia watchdog nenhum (a tabela 2.0.3 citava 5 min inexistentes): deadline no prefill e por token, default moderno de 30 min, `ADVPP_LLM_TIMEOUT_SECS` sobrescreve (`0` = sem teto), erro capturável via `Try/Catch`. Teste: `TestLlmTimeoutSeconds`.
- **Supressão de pensamento (técnicas do PiG em inferência crua).** `pkg/llm/thinking.go`: fechar `<think>` no prefill + remover blocos da saída; método `LLM:SuprimePensamento([lAtivo])`, default desligado. Testes: `TestCloseThinkingPrefill`, `TestStripThinking`.
- **Leitura de linha Q6_K, clamp de RoPE, HeadDim do tensor, atenção larga, fallback tied-embeddings, default `eps=1e-6`, `TensorNames()`, tokens especiais no Encode, flag `ComBOS`, infra QK-norm (qwen2/qwen3).** Robustez do motor para arquivos reais fora do par validado (ex.: MiniCPM5-1B, Llama-3.2-3B). Suite `pkg/llm` verde.

Fixed

- **Tabela de limites de recursos modernizada** (era 2.0.3): timeout do LLM agora real e configurável (ver acima).
- **GetEnv sem default retornava a string "Nil"** (de `ToString(nil)`), que `Empty()` não pega — fallback de PATH nunca acionava. Forma suportada: `GetEnv("VAR", "")`.

Em investigação

- **Qwen3 gera degenerado no motor nativo** (distribuição flat; referência dá Paris no mesmo prompt). Carrega sem crash; isolado com harness de logits, ablação de QK-norm, sweep de escala e leitura do graph no fonte do llama.cpp — dossiê em notas Joplin (`Qwen3 divergencia comprovada`). Llama-3.2-3B e MiniCPM íntegros e sem regressão.

[4.4.5] - 2026-10-04

Added

- **advplc rpo apo** — desmontagem de blobs APO extraídos do RPO: identificadores, literais, snippets de código e call-graph candidato, com relatório JSON/Markdown (`--catalog`, `--out`, `--format`). Parser do framing APO (magic `75 00 00 46 46` + kind `F / T`) validado contra 4 fixtures reais — 3.465/3.465 blobs do catálogo de teste (977.618 identificadores, 124.944 candidatos). Testes: `pkg/rpo/apo_blob_test.go`, `cmd/advplc/cmd_rpo_apo_test.go`.
- **advplc rpo pull** — extração ao vivo de recursos do RPO via sessão DAP (`GetApoRes`): lista por manifesto (`--manifest`) ou listing `--res`, saída em `--out`, senha via prompt ou `ADVPP_RPO_PASS` (nunca hardcoded). Smoke comprovado: recurso real baixado byte-a-byte idêntico (md5 conferido, evidência em `docs/rpo-evidence/rpo-pull-smoke.log`). Testes: `pkg/rpo/wire/*_test.go`, `pkg/rpo/dap/*_test.go`, `cmd/advplc/cmd_rpo_pull_test.go`.
- **Pacotes `pkg/rpo/wire` e `pkg/rpo/dap`** — codec de frames do protocolo do AppServer (`<IHIIH>`, magic `0xab21`, zlib opcional), handshake autenticado de 5 mensagens (validado ao vivo contra build `7.00.210324P`), banner de identificação, decode de resultado DAP (aspas/base64/ `NIL`); RPC stdio LSP/DAP com framing `Content-Length`, obtenção de token via Language Server e sessão de depuração (launch → breakpoints → stop → evaluate).
- **docs/RPO-EXTRACTION-METHODOLOGY.md** — documentação canônica das técnicas de extração RPO (T1–T14 com status de evidência, protocolo wire/DAP, métricas, gotchas e análise de gaps) + evidências curadas em `docs/rpo-evidence/`.

Fixed

- `advplc rpo inject` executava o bloco duas vezes (bloco órfão pós-switch removido). (2a3996a)
- Handshake do AppServer com deadline por passo (um deadline único de 1s estourava o RTT via proxy de captura). (a407683)

Security

- Credenciais do operador redigidas em documentos e comentários do repositório (`[REDACTED]`); senha do AppServer nunca presente em log ou artefato. (d7cc702)

[4.4.0] - 2026-10-03

Fixed

- **Isolamento entre sessões em banco remoto (multiempresa por `search_path`).** Cada conexão aberta por `DbConnection:Connect()` fica presa a uma conexão física (`SetMaxOpenConns(1)`): `SET search_path` passa a valer para todo comando seguinte daquela sessão — antes o pool do `database/sql` espalhava os comandos entre conexões. Conexão reposta pelo driver nasce em `public` (falha fechada: "tabela não existe"). Teste: `TestPinPool` . (073024c)
- **Cada VM usa a conexão que ela mesma abriu.** `DBSetDriver("TOPCONN")` pegava a conexão "ativa" global do processo — a da última sessão que conectou —, misturando sessões do `advplc serve`. Teste: `TestApplyRDDUsesOwnConnection` . Ao fim de cada sessão do `serve` as conexões dela são fechadas (`TestCloseOwnedConnections`). (247cb7b)
- **Requisição REST, `StartJob`, `FWJobStart`, `FWGridProcess`, gRPC e MCP ganham conexão própria.** A requisição REST compartilhava o engine do pai (requisições concorrentes trocavam a empresa umas das outras) e os jobs caíam no SQLite local vazio quando o banco era remoto. Agora cada VM-filha recebe um clone da configuração do pai, devolvido ao terminar. Testes: `TestChildVMClonesRemote`, `TestChildVMLocalEngineKeepsFactory`, `TestChildVMInheritsRDD`; prova ponta a ponta no GEBAN (`T_GEBISO`, 6 workers concorrentes em 2 empresas, Postgres real). (bfbda8c)
- **Sessão do `advplc serve` abandonada não segura mais a conexão.** Browser que recarrega ou fecha com menu/diálogo/browse aberto deixava a goroutine da sessão presa para sempre, com a conexão de banco dela. Agora, sem nenhum browser conectado por `ADVPP_WEBUI_ABANDON_SECONDS` (padrão 300), a sessão é encerrada e as conexões fechadas. Testes: `TestAbandonReleasesBlockedDialog`, `TestAbandonDropsOutput` .
- VM sem conexão própria fica no engine local (falha fechada) em vez de herdar a "ativa" global de outra sessão (`TestApplyRDDNoOwnConnDoesNotTakeOthers`); `EVAL` / `AEVAL` /ações de `MSDIALOG` reaproveitam a conexão do pai em vez de abrir uma por avaliação (`TestSharedChildVMDoesNotClone`); o estado `DB*` de cada VM-filha é liberado ao terminar (`TestChildVMDoneReleasesState`).

Added

- `ADVPP_MAX_REMOTE_CONNS` (padrão 50): limite de conexões remotas abertas por processo; acima dele, erro `limite de conexoes remotas atingido (N)` . Teste: `TestRemoteSlots` . (073024c)
- `FWMBrowse:SetMenuDef("")` deixa o browse somente leitura: a tela esconde Incluir/Editar/Excluir e o servidor recusa `save` / `delete` mesmo se o cliente enviar. Teste: `TestBrowseReadOnlyRejects` . (dc1152c)

[4.3.2] - 2026-10-01

Fixed

- `FWMBrowse` funcionava no SQLite e morria no Postgres remoto: `browseItems` / `browseSave` / `browseDelete` (`pkg/vm/browse.go`) endereçavam a linha pelo pseudo-campo `rowid`, que não existe no Postgres (nem no Oracle/MSSQL) — qualquer `Activate()` contra banco remoto abortava a sessão com erro. A coluna-chave agora é resolvida por `browseKeyColumn`: `R_E_C_N_O` (coluna real nos dois motores — no SQLite ela É o rowid) quando a tabela tem, `rowid` como fallback para tabelas sem `R_E_C_N_O` . Validado com `go test ./pkg/vm/ -run Browse` (7 PASS, 2 testes novos).

[4.3.1] - 2026-09-30

Added

- ReplaceAPO dinâmico com suporte a mudanças de tamanho
- Recriptografia automática após modificação

- Heurística aprimorada de identificação de ciphers
- Testes unitários para injector (7 testes)
- Makefile com targets RPO
- Scripts de automação (capture_and_inject.sh)

Changed

- Parser APO com validação melhorada
- Documentation atualizada
- Error handling aprimorado

Fixed

- Bug de compilação `-compile` (falta RPODB no INI)
- Problemas de slice bounds no parser
- Tratamento de edge cases na reconstrução do body

Performance

- Parse RPO 22KB: 40µs
- Parse RPO 9.5MB: 9.6ms
- Memória otimizada

[4.3.0] - 2026-09-30

Added

- Parser completo de estrutura RPO
- Parser de registros APO
- CLI `advplc rpo inject`
- Support para 24+ ciphers OpenSSL
- Sistema de captura LD_PRELOAD

Changed

- Refatoração do código para modularidade
- Melhorias na documentação

[4.2.0] - 2026-09-29

Added

- Heurística de identificação de cipher
- Mapeamento key/IV size → cipher
- Testes de heurística

[4.1.0] - 2026-09-28

Added

- Serializer JSON → binário
- Geração de bytcodes
- Comandos CLI básicos

[4.0.0] - 2026-09-26

Added

- Infraestrutura inicial
- Parser RPO básico

- Hook LD_PRELOAD
 - Testes unitários
-

Notas de Versão

4.3.1

- **Destaque:** ReplaceAPO dinâmico
- **Testes:** 17/17 passing
- **Performance:** <10ms para RPOs até 10MB

4.3.0

- **Destaque:** Parser completo
 - **Novo:** Suporte a múltiplos ciphers
 - **Testes:** Infraestrutura de testes
-

Changelog do projeto AdvPP